

Software Implementation and Testing Document

For

Group <20>

Version 1.0

Authors:

Alex Fenoga
Alejandro Valdes
Brian Nelson
Caleb Dindinger
Daniel Smith

1. Programming Languages (5 points)

Typescript: used for the front end of our application because it is statically typed and has native support in the Vue.js web framework.

C#: used for the backend of our application, chosen because we all share experience with it and ASP.NET is particularly strong for web applications.

PostgreSQL: used for our database; it integrates well with C# and it's a widely used relational database.

2. Platforms, APIs, Databases, and other technologies used (5 points)

ASP.NET Core is the backend framework used to handle server-side logic, and API requests - used in the C# portion of the project. Rest API is used to communicate with the front end and backend.

Vue.js is used for building our UI with a reactive and component-based approach.

PostgreSQL is the primary relational database for storing user data, and other necessary information. It is also used together with ASP.NET identity for user authentication.

Axios is used to make API requests from the client to the backend.

Docker is used to locally run both the database and the API server.

3. Execution-based Functional Testing (10 points)

The original Vue.js project was created with both unit testing (vitest) and E2E testing (playwright) in mind. However, due to time restrictions, these are not yet implemented in any meaningful way. Therefore, most of the client functional requirements testing was performed manually. For instance, to test user registration and login functionality, we would manually type in a username and password, observing the api request responses and visual changes to the web application.

The back end has integration tests set up for AuthenticationService. This includes a GitHub workflow that runs whenever someone opens a pull request or pushes changes to main that change the source code of the backend. We also used Scala to test new API endpoints for which there was no client implementation yet.

4. Execution-based Non-Functional Testing (10 points)

For NFR 1, we are using .NET Identity to handle authentication, including password hashing and salting. Nevertheless, we did check the database after creating a user account and found it to be stored as expected.

For NFR 3, we tested the application with Chrome v134 and Firefox v136 and found it to be working as expected.

NFR 4 and 5 were tested by using browser developer tools to inspect the response time for each request sent to the server and they were found to be well within requirements. Additionally, there are some integration tests for these endpoints and those also fell within requirements, even with

the additional time spent setting up the tests. Note that this may vary depending on environment, configuration, etc.

The features relevant to NFR 6 are currently not implemented and thus cannot be tested.

5. Non-Execution-based Testing (10 points)

In order to test the code that would be merged into the main repository, we referred to Brian or Alex depending on what code end we were trying to merge into. If we were working on the front end, it was reviewed by Alex as he is the most experienced with Vue. On the other hand, when we were working on the backend, Brian reviewed the code to assure that it was good to go. This is in addition to the self review each member of the group did themselves before creating a pull request.